



西安交通大学
XI'AN JIAOTONG UNIVERSITY

1

短距离无线通信专题实验报 告

学号:

姓名:

班级:

实验指导教师:

实验地点:

联系电话:

论文题目：2026 短距离无线通信专题实验

学生姓名：

指导教师：

摘 要

本实验报告基于 TI CC3200 微控制器平台，围绕“短距离无线通信”这一核心主题，系统地记录了从底层硬件控制到高层网络协议开发的一系列实验过程。报告详细阐述了 GPIO 中断控制、定时器 PWM 输出、I2C 传感器信息采集、Wi-Fi 基础连接（AP 与 STA 模式）、基于 Socket 的 TCP/UDP 网络传输等五大基础实验，以及基于综合传感器数据的网络通信实验。通过本次系列实验，深入理解了短距离无线通信（特别是 Wi-Fi 技术）的底层原理与应用层开发，掌握了 FreeRTOS 在物联网设备中的应用，为未来从事物联网与智能硬件的开发奠定了坚实基础。

关 键 词：CC3200；短距离无线通信；物联网；网络传输；传感器

Title: 2026 Short Distance Wireless Communication Special Experiment
name:
Supervisor:

ABSTRACT

This experimental report is based on the TI CC3200 microcontroller platform, focusing on the core theme of "short-range wireless communication". It systematically records a series of experimental processes from low-level hardware control to high-level network protocol development. The report elaborates on five basic experiments, including GPIO interrupt control, timer PWM output, I2C sensor information acquisition, Wi Fi basic connection (AP and STA modes), TCP/UDP network transmission based on Socket, as well as network communication experiments based on comprehensive sensor data. Through this series of experiments, we have gained a deep understanding of the underlying principles and application development of short-range wireless communication (especially Wi Fi technology), and mastered the application of FreeRTOS in IoT devices, laying a solid foundation for future development of IoT and intelligent hardware.

KEY WORDS: Keywords: CC3200; Short distance wireless communication; Internet of Things; Network transmission; Sensor

目 录

摘要.....	II
1 绪论.....	1
2 实验一：短距离无线通信简介及按键控制跑马灯.....	3
2.1 实验原理.....	3
2.2 实验内容.....	3
2.3 算法思路.....	3
2.4 关键代码分析.....	3
2.5 遇到的问题及解决办法.....	4
3 实验二：串口控制 LED 灯亮度.....	6
3.1 实验原理.....	6
3.2 实验内容.....	6
3.3 算法思路.....	7
3.4 关键代码分析.....	7
3.5 遇到的问题及解决办法.....	8
4 实验三：通过 I2C 读取传感器信息.....	9
4.1 实验原理.....	9
4.2 实验内容.....	9
4.3 算法思路.....	9
4.4 关键代码分析.....	10
4.5 遇到的问题及解决办法.....	11
5 实验四：WiFi 基础与基本连接.....	12
5.1 实验原理.....	12
5.2 实验内容.....	12
5.3 算法思路.....	13
5.4 关键代码分析.....	13
5.5 遇到的问题及解决办法.....	14
6 实验五：基于 WiFi 的 UDP、TCP 传输.....	15
6.1 实验原理.....	15
6.2 实验内容.....	15
6.3 算法思路.....	16
6.4 关键代码分析.....	16

6.5 遇到的问题及解决办法.....	17
7 实验六：网络通信基础及综合设计实验	18
7.1 实验原理.....	18
7.2 实验内容.....	18
7.3 算法思路.....	19
7.4 关键代码分析.....	19
7.5 遇到的问题及解决办法.....	20
8 结论与展望	22
致 谢	24

1 绪论

随着信息技术的飞速发展与第四次工业革命的深入推进，物联网（IoT, Internet of Things）已经从理论概念全面走向了实际应用的深水区，深刻地重塑着智能家居、工业制造、智慧医疗与自动驾驶等多个领域的面貌。在物联网的庞大体系中，短距离无线通信技术（如 Wi-Fi、Bluetooth、Zigbee 等）扮演着至关重要的“毛细血管”角色，它打破了传统线缆的物理空间束缚，实现了物理世界与数字空间的无缝对接。其中，Wi-Fi 技术凭借其极高的传输速率、无缝接入现有以太网基础设施的便利性以及广泛的终端普及率，成为了实现大带宽、多节点智能设备入网的首选方案。

在传统的物联网终端设计中，设备往往只扮演“数据搬运工”的角色，将所有原始数据打包上传至云端处理。然而，随着海量设备的接入，纯云端处理模式面临着网络带宽受限、通信延迟过高以及数据隐私泄露等严峻挑战。因此，“边缘计算”（Edge Computing）架构应运而生。边缘计算要求处于网络边缘的终端节点不仅具备数据采集与无线收发能力，还需要拥有足够的算力，能够在本地对多源异构数据进行实时过滤、融合计算与阈值判断，只将最有价值的告警信息或压缩后的数据上传。这种范式的转变，对现代微控制器的性能与网络通信机制提出了更高的要求。

本次“短距离无线通信实验”系列课程，正是紧贴上述宏观技术发展脉络，精选了德州仪器（TI）推出的业界首款具有内置 Wi-Fi 连网功能的单芯片微控制器——CC3200 作为核心实验平台。CC3200 芯片采用了极具创新性的“双核分离”物理架构：其一是基于 ARM Cortex-M4 内核的应用微控制器（MCU），运行频率达 80MHz，专用于处理用户复杂的业务逻辑与丰富的硬件外设（如 GPIO、UART、I2C、Timer 等）；其二是专门负责网络协议处理的网络处理器（NWP, Network Processor）。NWP 内部固化了完整的 802.11 b/g/n 射频基带、TCP/IP 网络协议栈以及强大的硬件加密引擎。这种架构极大地减轻了主 MCU 的计算负担，使得开发者在进行极其耗时的网络 Socket 通信时，依然能够保证应用层对底层传感器和执行器的微秒级实时响应，堪称开发高性能边缘计算节点的理想选择。

在软件生态与开发范式方面，本课程跳出了传统单片机“裸机前后台死循环”的思维局限。实验不仅引入了 Driverlib 硬件抽象层（HAL）库来规范底层硬件的操作，还在复杂的网络实验中全面引入了 FreeRTOS 实时操作系统以及 TI 封装的 SimpleLink API。通过掌握 RTOS 的多任务调度、中断上下文切换、钩子函数（Hook）调试以及网络事件回调机制，我们得以在单片机上实现全双工的非阻塞通信，构建出真正具备工业级稳定性的嵌入式软件架构。

本系列实验的设置遵循了“由浅入深、自底向上”的工程认知规律。实验一与实验二从最基础的 GPIO 中断控制流水灯和 UART 串口解析入手，掌握 CC3200 的硬件

基础与时钟配置；实验三引入 I2C 总线，攻克了 BMA222 三轴加速度传感器与 TMP006 温度传感器的数字通信协议；实验四与实验五则正式踏入网络大门，从 AP/STA 模式的配置，到最终实现基于 UDP/TCP 协议的客户端与服务器双向通信；最后的综合实验六，则要求我们将传感器采集、定时器硬件测距、边缘告警计算与异步非阻塞网络通信融为一体，完成一个具备实用价值的物联网环境监测终端的开发。

综上所述，本次短距离无线通信课程不仅仅是一次对电子信息专业理论知识的实操检验，更是一次全面提升系统级软硬件协同设计能力的工程历练。在当今“百年未有之大变局”与国家大力推进科技自立自强的背景下，扎实掌握从底层硬件驱动到上层网络协议的“全栈”核心技术，是我们当代电子信息学子顺应时代发展浪潮、为国家中高端制造与智能化转型贡献力量的必由之路。通过这一系列充满挑战与启发的实验，我不仅深化了对复杂嵌入式系统的理解，更为未来的科研与职业发展打下了坚实的技术底座。

2 实验一：短距离无线通信简介及按键控制跑马灯

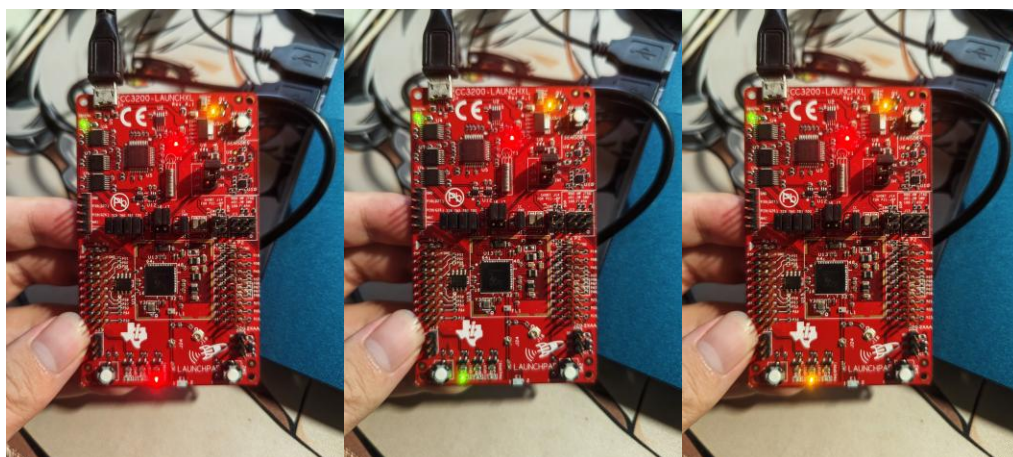
2.1 实验原理

短距离通信基础： 了解 Wi-Fi、蓝牙、ZigBee 等技术的速率与覆盖范围差异。

CC3200 中断系统与 GPIO： 键盘是由独立按键组成，按键按下与释放通过机械触点实现，具有抖动特性。CC3200 的 GPIO 支持可编程中断（屏蔽、边沿触发、电平触发）。利用中断机制，CPU 可以在外部事件（按键）发生时才进行处理，提高系统效率。

2.2 实验内容

通过 CC3200 板载的 SW2、SW3 按键控制三色 LED 的亮灭及跑马灯闪烁方向。按键采用中断方式进行检测，实现 LED 状态的动态切换。



2.3 算法思路

定义两个全局变量 `g_bLedRunning`（运行状态）和 `g_bDirection`（流水方向）。主程序初始化 GPIO 并注册 SW2、SW3 的下降沿中断。在主循环中，根据这两个状态标志位，依次点亮和熄灭红、橙、绿 LED。中断服务程序仅负责改变全局状态变量。

2.4 关键代码分析

2.4.1 按键中断处理代码

// SW3 (GPIO 13) 中断处理 - 切换运行/停止

```
void GPIOA1IntHandler(void) {  
    unsigned long ulStatus = MAP_GPIOIntStatus(GPIOA1_BASE, true);  
    MAP_GPIOIntClear(GPIOA1_BASE, ulStatus); // 清除中断标志，防止死循环
```

```

    if(!ulStatus & 0x20) {
        g_bLedRunning = !g_bLedRunning;
        if(!g_bLedRunning) GPIO_IF_LedOff(MCU_ALL_LED_IND);
    }
}

```

分析： 中断处理函数秉承“快进快出”的原则。首先读取并清除中断状态位，这是极其关键的一步，否则会引起中断嵌套导致死机。随后通过简单的异或操作反转运行状态标志。

2.4.2 GPIO 中断的注册与配置

// 注册 SW3 (GPIO 13) 中断

```

MAP_GPIOIntRegister(GPIOA1_BASE, GPIOA1_IRQHandler);
MAP_GPIOIntTypeSet(GPIOA1_BASE, 0x20, GPIO_FALLING_EDGE); // 下降沿触发
MAP_GPIOIntEnable(GPIOA1_BASE, 0x20);

```

分析： 这段代码是硬件中断的核心配置。通过 MAP_GPIOIntTypeSet 设定为 GPIO_FALLING_EDGE（下降沿），意味着只有在按键按下（电平由高变低）的瞬间才会产生一次中断，有效避免了长按导致的连续触发。

2.4.3 安全的 LED 状态切换逻辑

```

void SafeLedToggle(unsigned int led) {
    if(g_bLedRunning) {
        GPIO_IF_LedOn(led);
        MAP_UtIsDelay(8000000); // 软件延时
        GPIO_IF_LedOff(led);
    }
}

```

分析： 这里封装了点灯逻辑。在点亮和熄灭之间加入 MAP_UtIsDelay 形成视觉上的驻留时间。关键在于执行点亮前检查了 g_bLedRunning 标志位，确保在系统接收到停止指令时，流水灯能即刻中止，提高了系统的实时响应性。

2.4.4 跑马灯方向控制状态机

```

if(g_bDirection) { // 正向：红 -> 橙 -> 绿
    SafeLedToggle(MCU_RED_LED_GPIO);
    SafeLedToggle(MCU_ORANGE_LED_GPIO);
    // ...
} else { // 逆向：绿 -> 橙 -> 红
    SafeLedToggle(MCU_GREEN_LED_GPIO);
    // ...
}

```

分析： 主循环中的这段代码构成了一个简单的状态机。通过判断全局变量 g_bDirection（由 SW2 中断控制翻转），直接改变了代码的执行分支，从而实现了流水灯方向的即时反转。

2.5 遇到的问题及解决办法

- **问题：** 按键按下去一次，跑马灯状态切换了多次。

- **解决：** 机械按键存在抖动。虽然代码中仅配置了下降沿触发，但物理抖动会产生多个下降沿。解决办法是在软件中断中加入简单的延时过滤（如后续实验的 **Debounce** 机制），或在硬件回路上增加滤波电容。

3 实验二：串口控制 LED 灯亮度

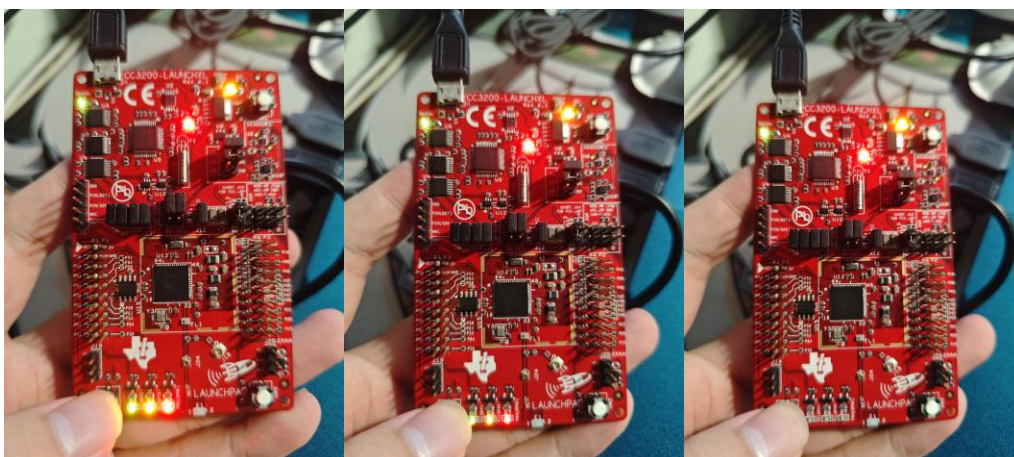
3.1 实验原理

定时器与 PWM： PWM（脉宽调制）通过改变方波的占空比来等效输出不同的模拟电压。CC3200 内部有 16 位通用定时器，在 PWM 模式下，定时器配置为向下计数，通过设定初始值（周期）和匹配值（改变占空比）即可产生 PWM 波来控制 LED 亮度。

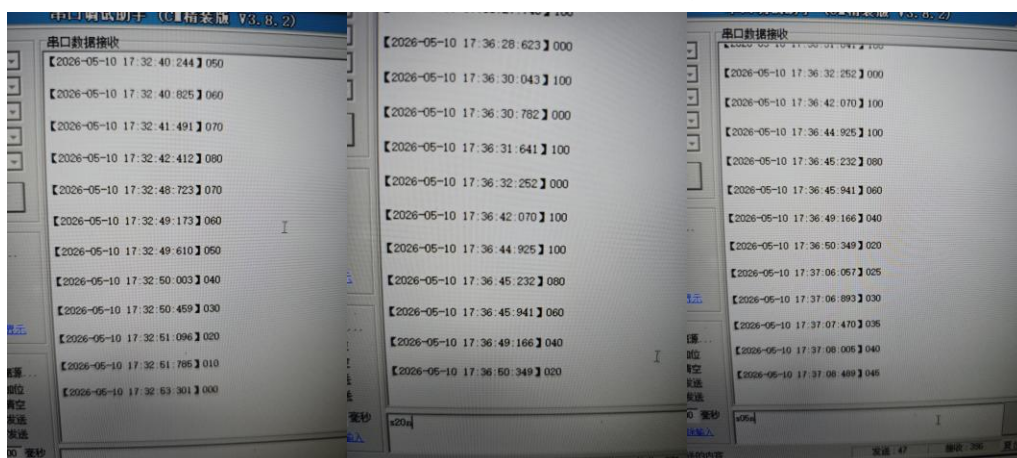
串行通信（UART）： 异步通信需要统一波特率、数据位和停止位。

3.2 实验内容

1. 通过定时器 PWM 控制 LED 亮度。
2. 实现 PC 机通过串口下发指令，控制 CC3200 板上的 LED 亮度。



3. 通过板载按键增加/减少 PWM 占空比，通过串口调节步进。



3.3 算法思路

利用全局变量 `g_iDutyCycle` 记录当前占空比。使能 UART 接收中断，当收到字符结尾为 'n' 时，解析前面的字符串并更新占空比。配置按键中断，按下 SW2/SW3 时增加/减少占空比，并利用 `MAP_UtilsDelay` 实现软件消抖。主循环检测到更新标志 `g_bUpdatePWM` 时，重载 Timer 匹配值。

3.4 关键代码分析

3.4.1 按 PWM 占空比更新逻辑

```
// 极端占空比处理与 Timer 匹配值计算
if (g_iDutyCycle <= 0) {
    // 0%时直接切换为 GPIO 模式并输出低电平，避免毛刺
    MAP_PinTypeGPIO(LED_RED_PIN, PIN_MODE_0, false);
    MAP_GPIOPinWrite(LED_GPIO_BASE, LED_GPIO_MASK, 0);
} else {
    // 恢复 Timer 模式并计算匹配值
    MAP_PinTypeTimer(LED_RED_PIN, PIN_MODE_3);
    unsigned long ulMatch = (ulReload * (100 - g_iDutyCycle)) / 100;
    MAP_TimerMatchSet(LED_RED_BASE, LED_RED_TIMER, ulMatch);
}
```

分析：此处的精妙之处在于对 0% 和 100% 占空比的特殊处理。由于硬件定时器在极值配置时可能仍会产生极窄的电平翻转（毛刺），代码通过直接切换引脚的复用模式（Pin Mux）为纯 GPIO 并拉低/拉高，彻底解决了 LED 微弱闪烁的问题。

3.4.2 严格的按键软件消抖机制

```
MAP_GPIOIntClear(GPIOA2_BASE, ulIntStatus2); // 1. 清除标志防止嵌套
MAP_UtilsDelay(900000);                       // 2. 延时约 30ms 避开抖动区
if(MAP_GPIOPinRead(GPIOA2_BASE, SW2_GPIO_PIN) == 0) // 3. 再次确认电平
{
    g_iDutyCycle += g_iStepSize;
    g_bUpdatePWM = true;
}
```

分析：这是极具工程实用价值的“三步消抖法”。先清中断位，再死区延时，最后再次采样引脚真实电平。这样彻底过滤了机械弹片接触瞬间产生的高频脉冲（毛刺），防止占空比数值出现异常跳变。

3.4.3 Timer 的 PWM 模式初始化配置

```
// 配置定时器分频与 PWM 模式
MAP_TimerConfigure(LED_RED_BASE, (TIMER_CFG_SPLIT_PAIR |
TIMER_CFG_B_PWM));
MAP_TimerLoadSet(LED_RED_BASE, LED_RED_TIMER, ulReload);
MAP_TimerEnable(LED_RED_BASE, LED_RED_TIMER);
```

分析：CC3200 的 16/32 位定时器非常灵活。此处使用 `TIMER_CFG_SPLIT_PAIR` 将其拆分为

两个独立的 16 位定时器，并设置为 `TIMER_CFG_B_PWM`（引脚复用 PWM 输出）。`ulReload` 决定了 PWM 的周期（频率），为后续修改 Match 值（占空比）提供了基准。

3.4.4 串口非阻塞接收与字符串解析

```
while(MAP_UARTCharsAvail(UARTA0_BASE)) {
    char cChar = MAP_UARTCharGetNonBlocking(UARTA0_BASE);
    if(cChar == 'n') { // 自定义结束符
        g_cCmdBuffer[g_ucCmdIdx] = '\0';
        // 解析步进值或占空比
        if(g_cCmdBuffer[0] == 's') g_iStepSize = atoi(&g_cCmdBuffer[1]);
        else g_iDutyCycle = atoi(g_cCmdBuffer);
        g_bUpdatePWM = true;
    } // ... 存入缓冲区
}
```

分析：串口中断处理中采用了 `MAP_UARTCharGetNonBlocking` 非阻塞读取。通过识别约定的结束符（'n'）来进行分包。利用 C 标准库 `atoi` 函数将 ASCII 字符串直接转换为整型数值，实现了 PC 端下发指令控制硬件参数的交互逻辑。

3.5 遇到的问题及解决办法

- **问题：** 按键触发时程序卡死或频繁触发。
- **解决：** 在 `GPIOIntHandler` 中使用了先清除标志位，再执行 `MAP_UtilsDelay(900000)` 约 30ms 的延时进行软件消抖，最后再次读取 `MAP_GPIOPinRead` 确认电平，有效去除了物理抖动。

4 实验三：通过 I2C 读取传感器信息

4.1 实验原理

I2C 总线： 双向、两线（SCL、SDA）、串行、多主控的同步通信总线。通信包含 START 信号、地址发送、ACK 应答、数据收发及 STOP 信号。

传感器原理： TMP006 是红外 MEMS 非接触式温度传感器；BMA222 是三轴加速度传感器，通过读取片内寄存器获取 X/Y/Z 轴的加速度数据（LSB 转化为物理量）。

4.2 实验内容

通过 I2C 读取 BMA222 和 TMP006 的数据。根据加速度信息判断设备的倾斜方向，并计算运动剧烈程度（加速度突变量）映射到红色 LED 的 PWM 亮度；根据温度高低调节 LED 的闪烁频率（温度越高闪烁越快），超过阈值在串口输出报警。



4.3 算法思路

初始化 I2C 外设、Timer（用于 PWM）。主循环中不断调用传感器驱动层的接口函数。对获取

的 XYZ 轴数据进行微分（与上一次值求差）求取突变和，乘以系数作为 PWM 占空比。温度值映射为 MAP_UtillsDelay 的参数，实现温度控制频率。

4.4 关键代码分析

4.4.1 数据处理与响应逻辑

```
// 计算 XYZ 轴的加速度突变（当前值与上一次值的差值绝对值之和）
float mutation = fabsf(ax - last_ax) + fabsf(ay - last_ay) + fabsf(az - last_az);
// 将突变量映射为占空比（变化量达到 10m/s^2 时占空比为 100%）
int dutyRed = (int)(mutation * 10);
if(dutyRed > 100) dutyRed = 100;
// 温度控制延时计算（防零除）
float temp_calc = (fTemp < 10.0f) ? 10.0f : fTemp;
unsigned long delay_val = (unsigned long)(200000000.0f / temp_calc);
MAP_UtillsDelay(delay_val);
```

分析：本段代码体现了物理量到控制量的映射思想。使用曼哈顿距离（绝对值之和）简化了加速度变化量的计算。温度控制延时的表达式巧妙地构建了反比关系，使得温度上升导致延时参数减小，进而提高了闪烁频率。

4.4.2 加速度重力分量解析与姿态判定

```
if(cAccZ > 0) {
    if(abs(cAccX) > abs(cAccY)) {
        if(cAccX > 20) direction = "向右倾斜";
        else if(cAccX < -20) direction = "向左倾斜";
    } // ...
} else if(cAccZ < -50) {
    direction = "倒置";
}
```

分析：充分利用了三轴加速度传感器（BMA222）受重力影响的特性。通过比较 X、Y、Z 三轴的分量大小，判断出重力矢量在设备坐标系中的主要指向，从而得出设备的物理姿态。逻辑分支清晰，无需复杂的三角函数运算。

4.4.3 占空比(Match 值)的安全换算

```
void SetPWMDuty(unsigned long base, unsigned long timer, int duty) {
    if(duty < 0) duty = 0;
    if(duty > 100) duty = 100;
    unsigned long match = (ulReload * (100 - duty)) / 100;
    MAP_TimerMatchSet(base, timer, match);
}
```

分析：对输入参数进行了强制的上下限幅（0~100），防止非法占空比导致定时器寄存器溢出。由于定时器在 PWM 模式下是向下计数的，占空比越大，意味着引脚保持高电平的时间越长，因此匹配值 match 需要用（100 - duty）进行反向换算。

4.4.4 RTOS 异常捕获钩子函数

```
void vApplicationStackOverflowHook(void *pxTask, signed char *pcTaskName) {  
    while(1);  
}  
void vApplicationMallocFailedHook(void) {  
    while(1);  
}
```

分析：引入 FreeRTOS 架构时必备的 Hook（钩子）函数。虽然函数体内只有 `while(1)` 死循环，但在实际开发中，如果任务栈溢出或堆内存分配失败，程序会定格在此处，此时通过 JTAG 仿真器连入即可迅速定位导致崩溃的具体任务，是关键 Debug 手段。

4.5 遇到的问题及解决办法

- **问题：**当温度过低时，除法运算可能导致延时过大甚至除以零异常。
- **解决：**引入了 `temp_calc` 变量，使用三目运算符设置了最低温度阈值限制（10.0f），增强了代码的鲁棒性。

5 实验四：WiFi 基础与基本连接

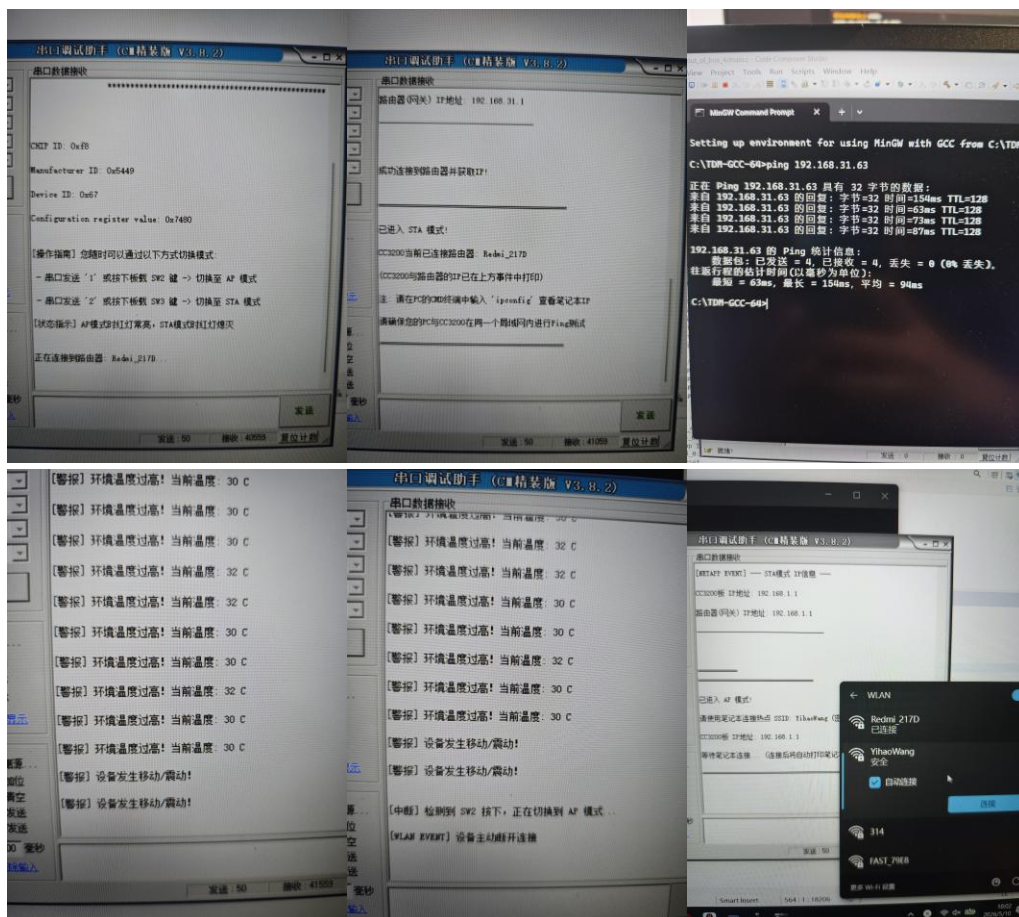
5.1 实验原理

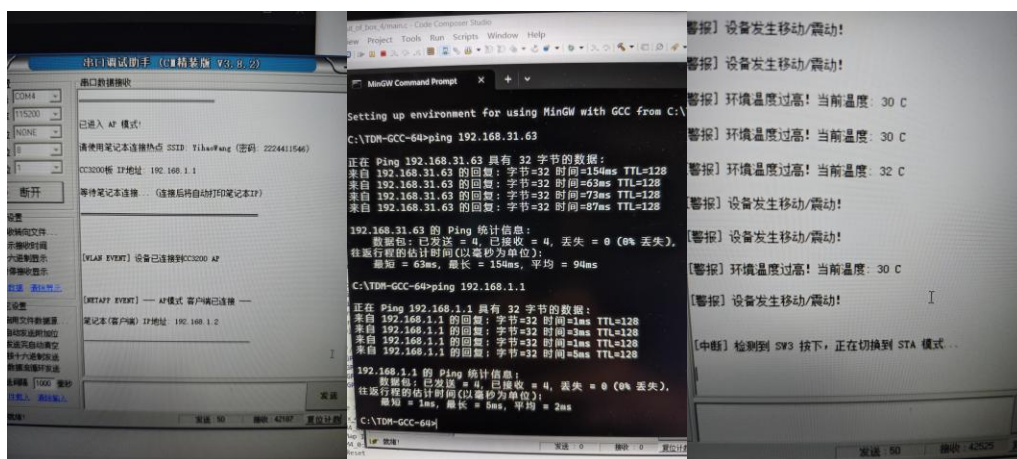
Wi-Fi 机制： 包含扫描（SCAN）、认证（Authentication）和关联（Association）阶段。CC3200 内置专用的网络处理器（NWP），负责 TCP/IP 栈及无线底层的 MAC 处理。

操作模式： 包含 STA（站点端，类似手机连路由）和 AP（热点端，自身发出 Wi-Fi）。

5.2 实验内容

将 CC3200 配置为 AP 或 STA 模式。PC 或手机连接该节点。实现通过串口按键（'1'或'2'）及硬件按键（SW2/SW3）在 AP 与 STA 模式之间热切换。并在连接成功后，读取传感器数据进行报警监控。





5.3 算法思路

引入 FreeRTOS 系统架构, 创建 OOBTask 作为主线程。定义全局变量 g_uiDeviceModeConfig 管理目标模式。封装 ConnectToNetwork() 函数, 在该函数内执行断开当前连接、停止 NWP、设置新模式、重启 NWP 的过程。主线程使用 osi_Sleep 释放 CPU 权限并周期性检测按键和传感器状态。

5.4 关键代码分析

5.4.1 网络模式切换逻辑

// 统一封装的网络模式切换函数 (增强稳定性)

```
long ConnectToNetwork() {
    if(IS_CONNECTED(g_ulStatus)) {
        sl_WlanDisconnect();
        osi_Sleep(2000); // 留出断开事件的处理时间
    }
    // NWP 重启使模式生效
    sl_Stop(SL_STOP_TIMEOUT);
    CLR_STATUS_BIT_ALL(g_ulStatus);
    osi_Sleep(2000);
    sl_Start(NULL, NULL, NULL);
    if(g_uiDeviceModeConfig == ROLE_AP) {
        // 等待 AP 模式下获取 IP 并打印
        while(!IS_IP_ACQUIRED(g_ulStatus)) osi_Sleep(10);
        // ...
    } else {
        ConnectToSTA(); // 执行 STA 连接流程
    }
}
```

分析: SimpleLink 的 NWP 在切换模式时必须严格遵循 sl_Stop 与 sl_Start 的重启流程, 且状态标志位的清零必须同步进行。代码中加入的 osi_Sleep(2000) 有效防止了因 NWP 响应不及时导致的硬件死锁。

5.4.2 WLAN 异步事件回调处理

```
void SimpleLinkWlanEventHandler(SlWlanEvent_t *pWlanEvent) {
    switch(pWlanEvent->Event) {
        case SL_WLAN_CONNECT_EVENT:
            SET_STATUS_BIT(g_ulStatus, STATUS_BIT_CONNECTION);
            // 提取并保存已连接的 SSID 和 BSSID(MAC)
            memcpy(g_ucConnectionSSID,
                pWlanEvent->EventData.STAandP2PModeWlanConnected.ssid_name, ...);
            break;
        // ...
    }
}
```

分析: CC3200 的 NWP (网络处理器) 与应用 MCU 通过异步事件通信。此回调函数独立于主线程运行, 当 Wi-Fi 底层完成连接后, 利用 SET_STATUS_BIT 设置标志位, 唤醒等待该事件的主循环, 并将路由器信息暂存在全局数组中供应用层使用。

5.4.3 IP 获取与不同模式的区别打印

```
case SL_NETAPP_IPV4_IPACQUIRED_EVENT:
    SET_STATUS_BIT(g_ulStatus, STATUS_BIT_IP_AQUIRED);
    UART_PRINT("CC3200 端 IP 地址: %d.%d.%d.%d\n\r",
        SL_IPV4_BYTE(pNetAppEvent->EventData.ipAcquiredV4.ip,3), ...);
// ...
case SL_NETAPP_IP_LEASED_EVENT:
    UART_PRINT("客户端(连接至 AP 的设备) IP 地址: %d.%d.%d.%d\n\r", ...);
```

分析: NetAppEventHandler 处理 IP 层事件。代码清晰区分了两种情况: STA 模式下, 设备自身获取到 IP 触发 IPACQUIRED 事件; 而 AP 模式下, 内部的 DHCP 服务器将 IP 分配给接入的手机/PC, 触发 IP_LEASED 事件, 体现了 CC3200 对不同网络拓扑的支持。

5.4.4 时序位移判定设备运动

```
iRet = BMA222ReadNew(&cAccXT1, &cAccYT1, &cAccZT1);
MAP_UtilsDelay((90*80*1000)); // 间隔采样
iRet = BMA222ReadNew(&cAccXT2, &cAccYT2, &cAccZT2);

sDelAccX = abs((signed short)cAccXT2 - (signed short)cAccXT1);
if(sDelAccX > csAccThreshold || ...) g_ucDryerRunning = 1;
```

分析: 实现移动侦测的一种低成本算法。无需复杂的卡尔曼滤波, 只需在短时间内进行两次采样求差值。一旦绝对差值超过设定的阈值 csAccThreshold, 即可判定设备在三维空间中发生了位置移动或振动, 从而更新状态标志。

5.5 遇到的问题及解决办法

- **问题:** 频繁切换 AP 和 STA 模式导致 SimpleLink 挂死, 无法获取 IP。
- **解决:** 在模式配置前后加入了延时, 并在重连前主动调用 sl_WlanDisconnect()。确保 SimpleLink 底层状态机彻底结束上一次连接后再开启下一次连接配置。

6 实验五：基于 WiFi 的 UDP、TCP 传输

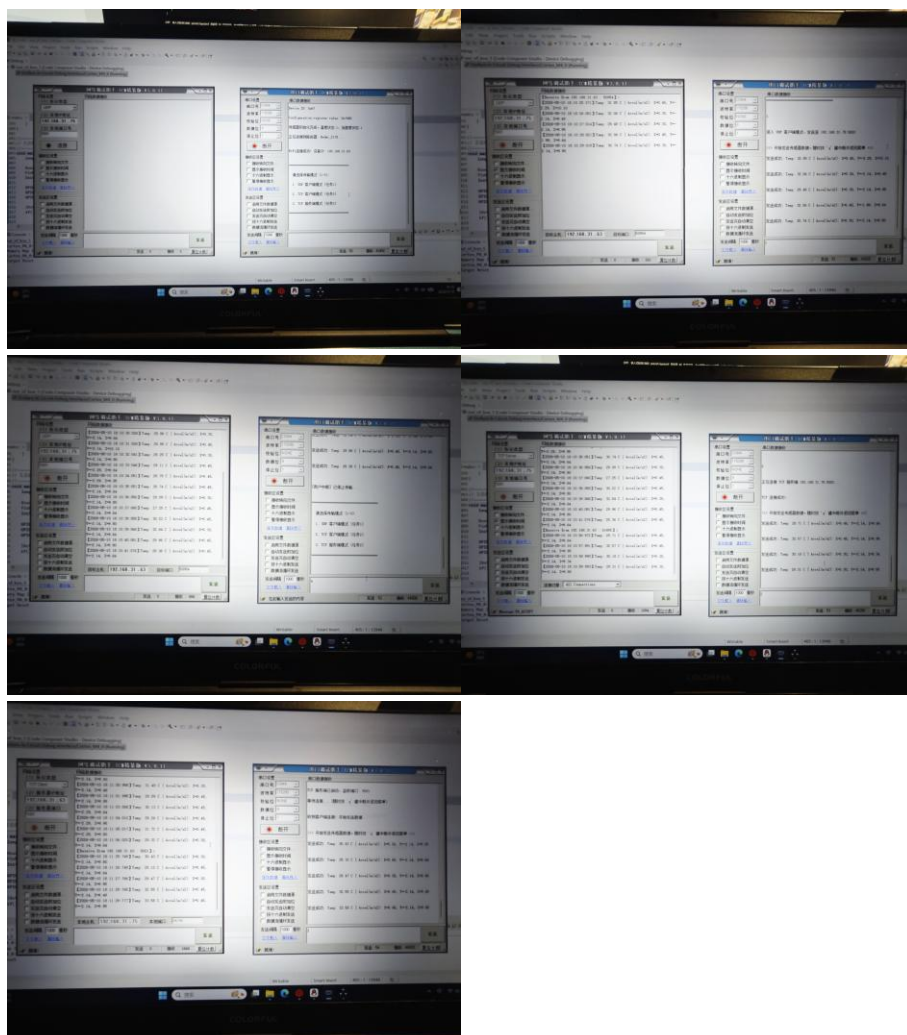
6.1 实验原理

TCP 与 UDP： TCP 是面向连接、可靠的数据流传输；UDP 是无连接、不可靠的数据报传输。

Socket 套接字： 操作系统封装 TCP/IP 协议族的抽象层。典型流程：TCP Server 需要 bind, listen, accept；TCP Client 需要 connect；UDP 则直接 sendto / recvfrom。

6.2 实验内容

在 STA 模式下连接至路由器。提供一个 UART 菜单，允许用户选择作为 UDP 客户端、TCP 客户端 或 TCP 服务器。建立连接后，循环读取温度和加速度数据，并将其格式化为字符串通过网络发送至 PC 端的网络调试助手，同时支持接收 PC 端指令。



6.3 算法思路

基于 FreeRTOS，创建 AppTask 处理网络逻辑。首先连接 Wi-Fi，成功后进入菜单循环。根据用户输入，进入相应的网络循环（如 RunTcpServer）。在数据发送循环中，获取传感器数据，拼接字符串，调用 sl_Send 或 sl_SendTo 发送数据，并在循环中非阻塞地检测串口是否有退出指令('q') 以便优雅地断开连接。

6.4 关键代码分析

6.4.1 TCP 服务器非阻塞 Accept 与数据发送

```
// TCP Server 模式配置
sl_SetSockOpt(iSockID, SL_SOL_SOCKET, SL_SO_NONBLOCKING, &enableOption,
sizeof(enableOption));
// ...
while(1) {
    iNewSockID = sl_Accept(iSockID, (struct SLSockAddr_t *)&sClientAddr, &iAddrSize);
    if(iNewSockID >= 0) {
        // 收到连接，进入发送数据的子循环
        SendSensorDataLoop(iNewSockID, NULL, 1);
        sl_Close(iNewSockID); // 结束时释放 Socket 资源
    } else if (iNewSockID == SL_EAGAIN) {
        osi_Sleep(10); // 非阻塞模式下无连接到达，让出 CPU
    }
}
```

分析：将 sl_Accept 设置为非阻塞模式是嵌入式网络编程的最佳实践之一。如果是阻塞模式，线程将挂起在此处，无法响应用户的串口退出指令或处理其他并发任务。SL_EAGAIN 的处理配合 osi_Sleep(10) 完美平衡了系统响应与 CPU 占用率。

6.4.2 手动实现浮点数转字符串

```
void FloatToStr(float val, char* str) {
    if (val < 0) {
        val = -val;
        sprintf(str, "-%.%02d", (int)val, (int)((val - (int)val) * 100));
    } else {
        sprintf(str, "%.%02d", (int)val, (int)((val - (int)val) * 100));
    }
}
```

分析：嵌入式标准 C 库的 printf/sprintf 若启用 %f 浮点支持，会大幅增加 ROM 占用量。这段代码通过分别提取浮点数的“整数部分”和“两位小数部分”来格式化字符串，既满足了传感器数据（温、湿度等）精度的要求，又极大地优化了内存空间。

6.4.3 UDP 客户端结构体装配与发送

```
SLSockAddrIn_t sAddr;
```

```
sAddr.sin_family = SL_AF_INET;
sAddr.sin_port = sl_Htons((unsigned short)g_uiPortNum); // 主机字节序转网络字节序
sAddr.sin_addr.s_addr = sl_Htonl(g_ulDestinationIp);
```

```
sl_SendTo(iSockID, g_cBsdBuf, strlen(g_cBsdBuf), 0, (SISockAddr_t*)sAddr, iAddrSize);
```

分析：规范的 BSD Socket UDP 发送流程。必须配置目标 IP 与端口。这里特别注意 `sl_Htons` 和 `sl_Htonl` 的调用，由于网络传输统一采用大端模式（Big-Endian），而 ARM 架构通常是小端模式（Little-Endian），这两句代码是保证对方正确解析 IP 和端口的核心步骤。

6.4.4 网络阻塞时的串口异步逃逸机制

```
for (i = 0; i < 100; i++) {
    if (MAP_UARTCharsAvail(CONSOLE)) {
        char c = MAP_UARTCharGetNonBlocking(CONSOLE);
        if (c == 'q' || c == 'Q') return -1; // 侦测到退出指令，中止网络发送
    }
    osi_Sleep(10); // 拆分延时
}
```

分析：在死循环中持续发送网络包时，系统很容易失去对外部指令的响应。代码中将本应为 1 秒的长延时，巧妙地拆分为 100 次 10 毫秒的短延时，并在每次短延时中非阻塞地查询串口是否有按键录入。这样既保证了数据发送的发送间隔，又实现了“随时按下‘q’键立刻退出”的优良交互体验。

6.5 遇到的问题及解决办法

- **问题：** C 语言在嵌入式中直接使用 `%f` 打印浮点数可能引发内存错误或不支持。
- **解决：** 自定义了 `FloatToStr` 转化函数，将浮点数放大 100 倍提取整数和小数部分进行字符串拼接，规避了标准库臃肿及指针错误的问题。

7 实验六：网络通信基础及综合设计实验

7.1 实验原理

超声波测距原理： 超声波模块（如 HC-SR04）的工作依赖于声波的反射机制。单片机通过 Trig 引脚输出至少 $10\mu\text{s}$ 的高电平脉冲触发测量，模块自动发送 40kHz 方波。回响引脚 Echo 会输出一个与超声波往返时间等宽的高电平信号。测距公式为： $\text{距离} = (\text{高电平时间} \times \text{声速}) / 2$ 。

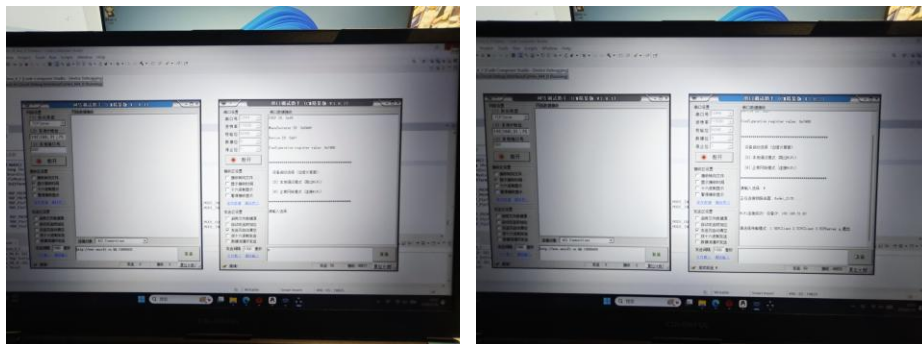
边缘计算（Edge Computing）架构： 物联网终端不仅负责采集数据，还具备本地逻辑判断能力。通过在硬件端（边缘侧）计算加速度差值、对比距离阈值，直接得出“振动报警”和“防碰撞报警”状态，再将高价值的告警信息与数据一同发送给 PC，可有效降低云端/PC 端的计算压力及网络延迟。

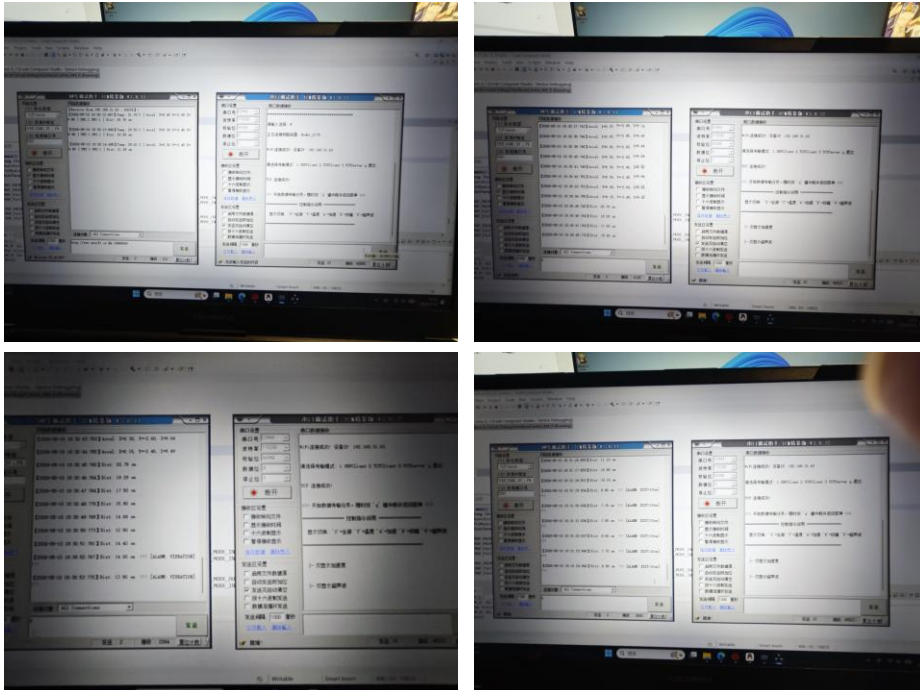
全双工非阻塞网络通信： 结合前述实验的 TCP/UDP Socket，使用非阻塞模式（SL_SO_NONBLOCKING）与软硬件定时器，在同一主循环内实现高频的命令接收与周期性的数据上报。

7.2 实验内容

开发一个具有边缘计算能力的综合物联网传感器节点。

1. 整合温度传感器、三轴加速度传感器、物理按键以及超声波传感器。
2. 建立交互式菜单，支持“本地调试模式”和“Wi-Fi 网络模式（TCP/UDP）”。
3. 实现边缘告警逻辑：当测距小于 10cm 或设备发生突变振动时，板载指示灯切换为橙色警告，并附加报警字符串发送。
4. 实现双向控制交互：PC 端可通过网络发送指令（如 'S', 'T', 'A', 'B', 'U'）来动态切换 CC3200 上报的数据类型（仅温度、仅加速度、仅距离或全部信息）。





7.3 算法思路

基于 FreeRTOS 创建一个主任务 AppTask。系统上电后，通过串口提供工作模式选择。若选择网络模式，则连接 Wi-Fi 并创建对应的非阻塞 Socket。核心循环 SendSensorDataLoop 以 100ms 为一个节拍 (Tick)。在每个 Tick 内：(1) 非阻塞地尝试从 Socket 读取网络指令，若有指令则更新显示模式变量 g_DisplayState。(2) 非阻塞地检测串口，若有 'q' 指令则断开连接退出。每经过 10 个 Tick (即 1 秒)，系统读取各传感器数据。根据定时器计数值计算出超声波距离；计算当前加速度与上一次记录值的差值判断振动。最后依据 g_DisplayState 的状态，将传感器信息和报警状态拼接为格式化字符串，通过网络 Socket 或本地串口输出。

7.4 关键代码分析

7.4.1 利用硬件定时器与软件超时机制实现超声波测距

// 等待回响高电平结束并获取定时器停止值

timeout = 20000000;

while(MAP_GPIOPinRead(GPIOA0_BASE, 0x8) != 0 && timeout > 0) timeout--;

ulStop = MAP_TimerValueGet(TIMERA0_BASE, TIMER_A);

if (timeout <= 0) return -2.0; // 超时保护

// 解决 32 位 向上计数定时器溢出翻转的问题

if (ulStop >= ulStart) ulDiff = ulStop - ulStart;

else ulDiff = (0xFFFFFFFF - ulStart) + ulStop + 1;

分析：在读取 Echo 高电平宽度的过程中，使用了 while 轮询电平。为防止前方无障碍物导致 Echo 永远为高或硬件掉线而引起的“死循环卡机”，代码中极具工程经验地加入了 timeout 递减机

制。同时，考虑到 32 位自由运行定时器可能在测量期间发生满溢清零（0xFFFFFFFF 翻转为 0），通过 if-else 分支妥善处理了定时器折返时的差值计算，确保了距离计算的绝对准确。

7.4.2 边缘计算与多源数据状态融合告警

```
// 加速度变化量超过阈值（如搬动、撞击）触发振动报警
int diffX = abs(cAccX - lastAccX); // ...计算 Y 和 Z 轴
if ((diffX > 20 || diffY > 20 || diffZ > 20) && tick_count > 0) {
    isVibrationAlarm = 1;
}
// 距离小于 10cm 触发距离报警
if (dist < 10.0f) isDistanceAlarm = 1;

// 状态融合控制与警报字符串拼接
if (isVibrationAlarm || isDistanceAlarm) {
    GPIO_IF_LedOn(MCU_ORANGE_LED_GPIO); // 点亮橙色报警灯
    GPIO_IF_LedOff(MCU_GREEN_LED_GPIO);
    strcpy(alarmMsg, " !!! [ALARM:");
    // ...拼接报警详情
}
```

分析：这段代码是“边缘计算”特性的直接体现。单片机并没有将庞大的原始传感器数据毫无保留地抛给网络，而是在本地（边缘侧）进行了过滤与阈值比对。一旦满足振动或防撞阈值，直接控制底层硬件报警（切换 LED 为橙色），并生成极简的报警标签下发给终端，极大提升了物联网系统的实时性与网络带宽的利用率。

7.4.3 非阻塞网络接收与动态指令解析

```
// 非阻塞接收网络端下发的指令
if (bIsTCP) recvLen = sl_Recv(iSockID, recvBuf, sizeof(recvBuf)-1, 0);
else recvLen = sl_RecvFrom(iSockID, recvBuf, sizeof(recvBuf)-1, 0, NULL, NULL);

// 如果收到数据，则根据指令修改设备上报状态
if (recvLen > 0) {
    if (strchr(recvBuf, 'S')) { g_DisplayState = 0; }
    else if (strchr(recvBuf, 'T')) { g_DisplayState = 1; }
    // ...处理其他字符
}
```

分析：此处通过 sl_SetSockOpt 提前将 Socket 配置为非阻塞模式。sl_Recv 调用时若无数据到达，会立刻返回 SL_EAGAIN，使得主循环代码可以继续向下执行去采集和发送数据。一旦解析到 PC 端发来的控制字符，即可修改全局状态变量 g_DisplayState，使设备从“盲目发送全部数据”智能切换到“按需发送指定数据”，实现了真正的全双工双向控制。

7.5 遇到的问题及解决办法

- **问题 1：** 超声波传感器无回波或连接线松动导致系统完全死机，网络连接断开。
 - **解决办法：** 超声波读取原本完全依赖引脚电平翻转的阻塞等待。在 ReadUltrasonicDistance() 函数中引入了 volatile int timeout 机制，强制在数千万

次空循环后超时退出，并返回特定的负数（如-1.0 或-2.0）作为错误码。主循环检测到负数时上报“Timeout/Err”，从而保护了 RTOS 整个任务队列免受死锁崩溃的影响。

- **问题 2：** 在同一线程中同时处理“等待网络数据(Recv)”和“发送传感器数据(Send)”时，数据上报存在严重的卡顿和延迟。
 - **解决办法：** 阻塞式的网络调用会挂起当前线程直到数据到来。解决方法是严格落实 Socket 的 `SO_NONBLOCKING` 非阻塞参数配置，同时利用 `tick_count % 10 == 0` 的计数器算法，实现了收、发速率的分离（接收循环 100ms 执行一次，发送每 1 秒执行一次），既保证了高频响应控制指令，又限制了发送频率防止占用过多网络带宽。
- **问题 3：** Wifi 使用计时器 A2 导致如果超声波使用 A2 计时会让 Wifi 无法连接。
 - **解决办法：** 修改为 A0 计时器来进行全局计时。

8 结论与展望

通过本次为期数周的短距离无线通信系列实验，我依托德州仪器（TI）的高性能 CC3200 微控制器平台，成功从零开始构建了一个兼具多维数据感知、边缘逻辑计算以及全双工网络通信能力的综合物联网终端节点。在此过程中，我的工程开发能力和对底层计算机系统的理解得到了跨越式的提升。

在硬件外设与底层驱动层面，我彻底走出了只依赖高级库调用而不知其所以然的误区。从使用 GPIO 中断与精准延时实现带消抖的按键检测，到利用 16/32 位定时器配置 PWM 波形平滑控制 LED 亮度；从解析 I2C 总线时序以稳定读取 BMA222 三轴加速度传感器与 TMP006 温度传感器的内部寄存器，到巧妙利用定时器输入捕获机制精确测量超声波传感器的高电平脉宽从而推算物理距离。这些实操让我熟练掌握了嵌入式系统与外部物理世界交互的数字接口规范。

在软件架构与网络通信层面，本次实验最大的突破在于跨越了传统单片机“裸机前后台轮询”的粗放模式。得益于 CC3200“应用 MCU+网络处理器”的双核异构架构，配合 FreeRTOS 实时操作系统与 SimpleLink 驱动库，我深刻体会到了异步事件回调与操作系统调度的魅力。在最终的综合设计中，我成功利用 SL_SO_NONBLOCKING（非阻塞 Socket）配合多重软硬件定时计数器，将阻塞式的网络收发与本地高频的数据采集解耦，完美实现了一个无卡顿、全双工运行的物联网系统。同时，“边缘计算”概念的引入是本次实验的点睛之笔：系统没有将海量传感器数据盲目抛给网络，而是在 MCU 端实时比对超声波防撞阈值，并对加速度的时序差值进行微分计算以判定异常振动。多源状态在本地融合后，直接驱动 LED 物理报警并上报极其精简的告警报文，极大降低了网络带宽负担并提升了系统的实时响应能力。

尽管当前的物联网边缘节点已经具备了较为完善的基础功能，并在实验室环境下表现出良好的稳定性，但若要将其推向实际的工业化或商业化应用，系统仍存在广阔的改进与升级空间。

首先是功耗管理与续航问题。当前的程序循环一直在高频运行（Active Mode），并未充分利用 CC3200 强大的电源管理模块（PRCM）。未来的优化方向是引入事件驱动的低功耗架构，在两次传感器采样的间隔期或网络空闲时，让 MCU 进入休眠（LPDS）甚至冬眠（Hibernation）模式，仅通过底层网络处理器维持 Wi-Fi 连接或由外部 GPIO 引脚唤醒，这将使设备在电池供电下运行数月甚至数年成为可能。

其次是网络协议的高级化与安全性增强。目前系统采用的是原生 TCP/UDP 裸套接字通信，这种方式在局域网内虽然高效，但在复杂的广域网环境下面临丢包、连接易断以及数据明文传输被窃听的风险。下一步，应当引入 MQTT、CoAP 等专门为物联网设计的轻量级发布/订阅协议，以增强网络传输的 QoS 保障；同时，充分调用 CC3200 硬

件加密引擎开启 TLS/SSL 安全套接字，为设备建立金融级的加密通信隧道。

最后，在边缘计算的深度上，当前仅仅停留在简单的阈值比较层面。随着微型机器学习（TinyML）的发展，未来完全可以在 Cortex-M4F 内核中部署轻量级的神经网络模型，让单片机不仅能判断“是否振动”，更能精准识别出设备处于“跌落、持续摇晃、抑或是机器正常运转”的具体姿态和工况。

回顾整个实验历程，从最初点亮一颗 LED 的喜悦，到最终看着格式化的传感器报文在 TCP 服务器上流畅滚动，一行行代码凝聚了无数次 Debug 的汗水与对未知的探索。短距离无线通信技术只是物联网庞大冰山的一角，但这次实验为我打下了坚实的全栈软硬件底层基石。在未来的科研与职业生涯中，我将带着这份严谨、务实且勇于创新的工程精神，继续在万物互联的星辰大海中破浪前行。

致 谢

致谢：本次“短距离无线通信实验”系列课程能够顺利完成，离不开诸多师长的悉心指导和同学们的无私帮助。在此，我谨向所有给予我支持与鼓励的人表达最诚挚的谢意。

首先，我要特别感谢本课程的指导教师**陈伟老师**。陈老师不仅在专业技术上深入浅出地为我们剖析了 CC3200 单片机底层原理、Wi-Fi 通信机制以及 FreeRTOS 等硬核知识，精心准备了图文并茂的 PPT 与参考例程；更在开课之初，以“百年未有之大变局”和“中华民族伟大复兴”的宏观视角，深刻分析了国内外科技领域的博弈与发展态势，勉励我们要在时代的浪潮中“顺势而为”。这番教诲不仅极大地激发了我的专业自豪感，更为我未来的学业与职业规划指明了方向。在实验调试过程中遇到瓶颈（如 SimpleLink 状态机卡死、底层硬件抖动等问题）时，老师的耐心解答也让我茅塞顿开。

其次，感谢学校**电信学部信通学院**为我们提供了 CC3200 实验板卡、丰富的传感器模块以及优良的实验室环境，使我们能够将纸面的理论知识转化为看得见、摸得着的工程实践。

最后，感谢这段充实而宝贵的实验经历。个人的成长离不开国家的强盛，正如老师所言，当前是对自己未来最佳投资的时期。在未来的学习与科研道路上，我将铭记师长教诲，扎实电子与信息技术的专业技能，努力在国家中高端制造与智能化的科技突破中，贡献属于自己的一份力量。